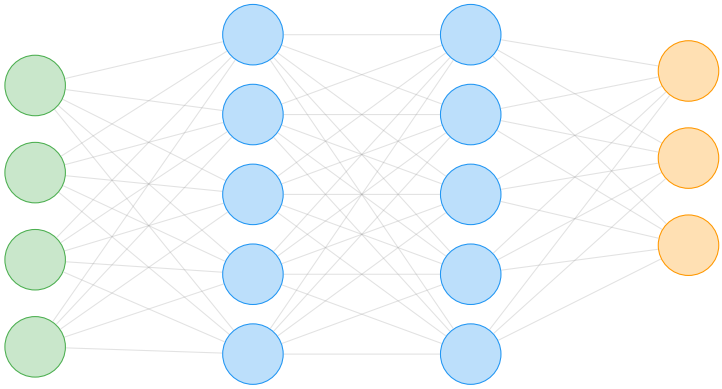
神经网络：面向人工智能专业学生的综合教材

面向人工智能专业学生的综合指南



教材

G. Qu et al.

指导教师：G Qu. et al.

云南大学

智能科学与技术系

中国

三月 2026

License

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.

About This Book

This comprehensive textbook on Neural Networks is designed for students and practitioners in Artificial Intelligence. It covers the fundamental concepts, mathematical foundations, and practical implementations of modern neural network architectures.

Main Features:

- Comprehensive coverage from basic perceptrons to advanced transformers
- Detailed mathematical derivations and explanations
- Practical examples using PyTorch framework
- GPU acceleration and CUDA programming concepts
- Real-world applications and case studies

Authors: G. Qu et al.
Supervisor: G Qu. et al.
Institution: 云南大学, 智能科学与技术系
Year: 2026

摘要

神经网络彻底改变了人工智能领域，使计算机视觉、自然语言处理、语音识别等众多领域取得了突破性进展。本教材全面介绍神经网络的基础知识，从基本概念到最先进的网络架构。

本书共十一章，内容包括：

1. **数学基础：** 涵盖理解神经网络所需的线性代数、微积分和概率论基础知识。
2. **激活函数：** 对Sigmoid、Tanh、ReLU及各种高级激活函数进行全面分析，并提供选择指南。
3. **损失函数：** 详细介绍交叉熵、均方误差及各类任务专用损失函数。
4. **神经网络架构：** 深度神经网络(DNN)、卷积神经网络(CNN)和循环神经网络(RNN)。
5. **核心操作：** 卷积、池化和归一化操作，配有严格的数学推导。
6. **高级机制：** 跳跃连接和通道注意力机制，包括残差学习。
7. **优化：** 梯度下降算法、各类优化器（SGD、Adam、AdamW）和学习率调度器。
8. **经典架构：** 从LeNet和AlexNet到ResNet、VGG及Transformer架构。
9. **计算分析：** 网络效率分析的参数量统计和FLOPs计算。
10. **训练与框架：** 实际训练过程、超参数调优、PyTorch实现及CUDA编程概念。
11. **数据集处理：** 数据收集、标注、增强和划分策略。

每章都包含详细的数学推导、实际案例以及对经典和最新研究论文的参考文献。完成本教材学习后，读者将具备设计、实现和训练各类神经网络应用的能力。

关键词： 神经网络、深度学习、机器学习、CNN、RNN、Transformer、优化、反向传播、PyTorch、计算机视觉、自然语言处理

Acknowledgements

We would like to express our sincere gratitude to all those who have contributed to the development of this textbook.

First and foremost, we thank the pioneering researchers whose groundbreaking work laid the foundation for modern neural networks, including Yann LeCun, Geoffrey Hinton, Yoshua Bengio, and many others whose contributions have shaped the field of deep learning.

We are grateful to our colleagues at 云南大学 for their valuable feedback and suggestions during the development of this material.

Special thanks to the open-source community, particularly the developers of PyTorch, NumPy, and other tools that have made deep learning accessible to researchers and practitioners worldwide.

Finally, we thank our families and friends for their patience and support throughout this project.

The Authors
三月 2026

To all the curious minds who seek to understand
the mathematics of learning machines,
and to the future researchers who will push
the boundaries of artificial intelligence.

- Authors -

Contents

Table of contents 9

List of figures 11

List of tables 13

List of listings 15

1 数学基础 1
- 1.1 引言 1
- 1.2 线性代数 1
  - 1.2.1 向量和矩阵 1
- 1.3 微积分 2
  - 1.3.1 导数和梯度 2
- 1.4 概率论 2
  - 1.4.1 Softmax分布 2
- 1.5 优化 3
- 1.6 信息论 3
- 1.7 本章小结 3

2 激活函数 5
- 2.1 引言 5
- 2.2 Sigmoid函数族 5
  - 2.2.1 Sigmoid（逻辑函数） 5
  - 2.2.2 双曲正切（Tanh） 6
- 2.3 ReLU函数族 6
  - 2.3.1 ReLU 6
  - 2.3.2 Leaky ReLU 7
  - 2.3.3 GELU（高斯误差线性单元） 7
- 2.4 Softmax函数 7
- 2.5 选择激活函数 7
- 2.6 本章小结 7

3 损失函数 9
- 3.1 引言 9
- 3.2 回归损失函数 9
  - 3.2.1 均方误差（MSE） 9
  - 3.2.2 平均绝对误差（MAE） 10
- 3.3 分类损失函数 10
  - 3.3.1 二元交叉熵 10
- 3.4 高级损失函数 10
  - 3.4.1 Focal Loss 10
  - 3.4.2 标签平滑 10
- 3.5 选择损失函数 11
- 3.6 本章小结 11

4 神经网络架构 13
- 4.1 引言 13
- 4.2 深度神经网络（DNN） 13

4.3 卷积神经网络（CNN） 13
4.4 循环神经网络（RNN） 14
4.5 长短期记忆网络（LSTM） 14
4.6 门控循环单元（GRU） 14
4.7 架构选择 14
4.8 本章小结 14

5 神经网络核心操作 17
5.1 引言 17
5.2 卷积操作 17
5.3 池化操作 17
5.3.1 最大池化 17
5.3.2 平均池化 18
5.4 归一化技术 18
5.4.1 批归一化（BatchNorm） 18
5.4.2 层归一化（LayerNorm） 18
5.5 本章小结 18

6 跳跃连接与注意力机制 21
6.1 跳跃连接 21
6.1.1 残差连接（ResNet） 21
6.2 注意力机制 21
6.2.1 自注意力 21
6.3 本章小结 22

7 优化器与学习率调度器 23
7.1 梯度下降优化 23
7.1.1 随机梯度下降（SGD） 23
7.2 自适应学习率方法 23
7.2.1 Adam（自适应矩估计） 23
7.3 AdamW（带权重衰减的Adam） 24
7.4 学习率调度 24
7.5 优化器选择 24
7.6 本章小结 25

8 经典网络架构 27
8.1 神经网络演进 27
8.2 LeNet（1998） 27
8.3 AlexNet（2012） 27
8.4 VGGNet（2014） 28
8.5 GoogLeNet / Inception（2014） 28
8.6 ResNet（2015） 28
8.7 Transformer（2017） 28
8.8 本章小结 29

9 参数量与计算量分析 31
9.1 模型效率的重要性 31
9.2 参数计数 31
9.2.1 全连接层 31
9.2.2 卷积层 31
9.3 FLOPs（浮点运算次数） 32
9.3.1 卷积层FLOPs 32

9.4 实际示例 32
9.5 迁移学习与效率 32
9.6 本章小结 33

10 训练过程与框架 **35**
10.1 训练循环 35
10.2 超参数 35
10.2.1 学习率 35
10.2.2 批次大小 35
10.3 正则化 35
10.3.1 Dropout 36
10.4 反向传播 36
10.5 PyTorch框架 36
10.6 CUDA与GPU训练 36
10.7 本章小结 38

11 数据集处理 **39**
11.1 数据收集 39
11.1.1 数据来源 39
11.2 数据标注 39
11.2.1 标注类型 39
11.3 数据划分 39
11.3.1 基本划分 39
11.4 数据增强 40
11.4.1 图像增强 40
11.5 数据加载 40
11.6 归一化 41
11.7 本章小结 41

List of Figures

List of Tables

2.1 激活函数选择指南 8

3.1 损失函数选择指南 11

4.1 架构选择指南 15

5.1 归一化方法比较 18

7.1 优化器选择指南 24

8.1 架构比较 29

9.1 AlexNet参数统计 32

10.1 超参数范围 35

Listings

Chapter 1

数学基础

1.1 引言

神经网络建立在坚实的数学基础之上，结合了线性代数、微积分、概率论和优化理论。本章提供理解神经网络学习和预测所需的基本数学知识。我们将从深度学习应用的角度介绍这些主题，提供理论见解和实践直觉。

神经网络的数学可以看作是线性变换（矩阵乘法）与非线性激活函数的组合，通过基于梯度的优化方法进行优化。理解这个框架需要熟练掌握向量和矩阵运算、多元函数求导以及随机优化。

1.2 线性代数

线性代数提供了在神经网络中表示和操作数据的语言。神经网络基本上通过线性变换（矩阵乘法）结合非线性激活函数来运算。

1.2.1 向量和矩阵

向量是有序的数字集合，可以表示n维空间中的一个点。在神经网络中，输入数据、隐藏表示和参数都表示为向量或更高维的张量。

定义 1 (向量). 向量 $\mathbf{x} \in \mathbb{R}^n$ 是 n 个实数的有序元组：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = (x_1, x_2, \ldots, x_n)^\top \tag{1.1}$$

其中 x_i 是向量的第 i 个分量。

定义 2 (矩阵). 矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 是 m 行 n 列的二维数组：

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \tag{1.2}$$

其中 a_{ij} 表示第 i 行第 j 列的元素。

例 1 (神经网络层作为矩阵运算). 单层全连接神经网络的运算：

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{1.3}$$

其中：

- $\mathbf{x} \in \mathbb{R}^{d_{in}}$ 是输入向量

- $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ 是权重矩阵
- $\mathbf{b} \in \mathbb{R}^{d_{out}}$ 是偏置向量
- $\mathbf{y} \in \mathbb{R}^{d_{out}}$ 是输出向量

如果 $d_{in} = 784$（MNIST图像）且 $d_{out} = 128$，则 $\mathbf{W}$ 包含 $784 \times 128 = 100,352$ 个参数。

1.3 微积分

微积分，特别是微分学，对于理解神经网络通过基于梯度的优化进行学习至关重要。

1.3.1 导数和梯度

定义 3 (梯度). 对于标量函数 $f : \mathbb{R}^n \to \mathbb{R}$，梯度是偏导数的向量：

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \tag{1.4}$$

梯度指向函数最速上升的方向。

性质 1 (链式法则). 反向传播中，我们计算标量输出（如损失）相对于向量输入的梯度：

$$\frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\top \frac{\partial L}{\partial \mathbf{y}} = \mathbf{J}^\top \nabla_{\mathbf{y}} L \tag{1.5}$$

后向传播高效地计算这个乘积，而无需显式形成完整的雅可比矩阵。

1.4 概率论

概率论为理解不确定性、进行预测和设计损失函数提供了框架。

1.4.1 Softmax分布

Softmax函数将logit向量 $\mathbf{z} \in \mathbb{R}^K$ 转换为概率分布：

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{1.6}$$

属性：

- $\sum_{i=1}^{K} \text{softmax}(z_i) = 1$
- $\text{softmax}(z_i) \in (0, 1)$ 对所有 i
- $\arg\max_i \text{softmax}(z_i) = \arg\max_i z_i$

用于多类分类输出。

1.5 优化

神经网络训练本质是一个优化问题：找到最小化损失函数的参数。

问题 1 (神经网络训练). 给定：

- 带参数 θ 的神经网络 $f(\mathbf{x};\theta)$
- 数据集 $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$
- 损失函数 $\mathcal{L}(\theta)$

找到：

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta) \tag{1.7}$$

其中通常：

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^N \ell(f(\mathbf{x}_i;\theta), \mathbf{y}_i) \tag{1.8}$$

1.6 信息论

信息论为理解损失函数（特别是交叉熵）提供了框架。

定义 4 (KL散度).

$$D_{KL}(p\|q) = \sum_x p(x)\log\frac{p(x)}{q(x)} \tag{1.9}$$

KL散度衡量从 q 到 p 的"距离"（不是真正的度量，因为不对称）。

例 2 (交叉熵损失). 对于 N 个样本和 K 个类的分类：

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^N\sum_{k=1}^K y_{ik}\log\hat{y}_{ik} \tag{1.10}$$

其中 y_{ik} 是真实标签（独热编码），$\hat{y}_{ik} = \text{softmax}(z_{ik})$ 是预测。

最小化交叉熵等价于最小化真实分布和预测分布之间的KL散度。

1.7 本章小结

本章涵盖了神经网络的基本数学基础：

1. **线性代数：** 向量、矩阵、张量运算和范数是神经网络中表示和操作数据和参数的构建块。
2. **微积分：** 导数、梯度和链式法则是反向传播计算梯度的基本工具。
3. **概率论：** 分布、期望和贝叶斯公式为理解预测和设计损失函数提供了框架。
4. **优化：** 梯度下降及其变体是神经网络训练的核心。
5. **信息论：** 熵和KL散度证明了交叉熵损失的使用是合理的。

Chapter 2

激活函数

2.1 引言

激活函数是应用于神经网络中每个神经元（或层）输出的数学函数。它们向网络引入**非线性**，使其能够学习线性函数无法捕获的复杂模式。

激活函数的选择显著影响：

- 网络学习复杂模式的能力
- 训练稳定性和收敛速度
- 反向传播中的梯度流动
- 神经元的输出范围
- 计算效率

2.2 Sigmoid函数族

Sigmoid函数将输入压缩到范围 $(0, 1)$。

2.2.1 Sigmoid（逻辑函数）

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \tag{2.1}$$

属性：

- **输出范围：** $(0, 1)$
- **导数：** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- **可解释为概率**

优势 1.

平滑梯度

输出可解释为概率

历史上广泛使用

劣势 1.

梯度消失： 对于大的 $|x|$，$\sigma'(x) \approx 0$

非零中心： 输出总是正的

饱和： 在0和1处饱和

2.2.2 双曲正切（Tanh）

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1 \tag{2.2}$$

属性：

- 输出范围： $(-1, 1)$
- 导数： $\tanh'(x) = 1 - \tanh^2(x)$
- 零中心输出

比Sigmoid更强的梯度，通常是隐藏层的更好选择。

2.3 ReLU函数族

ReLU函数族因计算效率高和训练深度网络的有效性而成为现代深度学习的主流选择。

2.3.1 ReLU

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{2.3}$$

性质列表 1 (ReLU属性).

- 输出范围： $[0, \infty)$
- 导数： $\mathrm{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$
- 计算高效
- 稀疏激活

优势 2.

计算非常快

减少正激活的梯度消失问题

稀疏表示

深度网络的出色经验性能

劣势 2.

Dying ReLU问题： 神经元可能"死亡"

非零中心

输出无界

[Dying ReLU问题] 如果神经元的权重更新使得所有输入都满足 $\mathbf{w}^\top \mathbf{x} + b \leq 0$，神经元将永远输出零。这被称为"死亡ReLU"问题。

2.3.2 Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} = \max(0, x) + \alpha \min(0, x) \tag{2.4}$$

其中 α 是一个小正数（通常 $\alpha = 0.01$）。

解决了Dying ReLU问题，同时保持ReLU的优点。

2.3.3 GELU（高斯误差线性单元）

$$\text{GELU}(x) = x \cdot \Phi(x) \tag{2.5}$$

其中 $\Phi(x)$ 是标准正态分布的CDF。

GELU用于BERT、GPT和许多Transformer架构。

性质列表 2 (GELU属性).

- **随机正则化器**
- **所有区域非线性**
- **接近零中心**

2.4 Softmax函数

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}} \quad \text{for } i = 1, \ldots, K \tag{2.6}$$

Softmax用于多类分类输出层。

属性：

- **输出范围：** 每个输出在 $(0, 1)$
- **和约束：** $\sum_i \text{softmax}(x_i) = 1$
- **概率解释**

2.5 选择激活函数

2.6 本章小结

1. **激活函数引入非线性**，使网络能够学习复杂模式。
2. **Sigmoid和Tanh** 将输入压缩到有界范围，但存在梯度消失问题。
3. **ReLU** 计算效率高，减少梯度消失，但可能存在死亡神经元问题。
4. **Leaky ReLU、PReLU和ELU** 解决了Dying ReLU问题。
5. **GELU** 在Transformer架构中变得流行。
6. **Softmax** 用于多类分类输出层。
7. **选择取决于** 任务、架构和训练稳定性。

Table 2.1 – 激活函数选择指南

层类型	推荐函数	备注
隐藏层（通用）	ReLU, GELU	ReLU是默认选择
隐藏层（深度）	ReLU, GELU, ELU	避免深度网络中的sigmoid/tanh
隐藏层（**RNN**）	Tanh	通常是RNN的默认值
隐藏层（**Transformer**）	GELU	BERT、GPT等使用
输出（二分类）	Sigmoid	概率输出
输出（多分类）	Softmax	概率分布
输出（回归）	Identity / 无	无界输出

Chapter 3

损失函数

3.1 引言

损失函数（也称为目标函数、成本函数或误差函数）是量化神经网络预测与真实目标匹配程度的数学度量。训练过程旨在通过基于梯度的优化调整网络参数来最小化这个损失。

损失函数应该：

1. 可微（用于基于梯度的优化）
2. 适当地惩罚较大误差
3. 鼓励特定任务的正确行为
4. 具有有用的统计解释

3.2 回归损失函数

回归任务预测连续值。

3.2.1 均方误差（MSE）

Mean Squared Error（L2损失）：

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 \tag{3.1}$$

属性：

- **凸函数（线性模型）**
- **对异常值敏感**
- **假设高斯误差分布**

优势 3.

凸函数，易于优化

闭式解（线性回归）

平滑梯度

劣势 3.

对异常值敏感

二次惩罚可能导致大误差主导

3.2.2 平均绝对误差（MAE）

Mean Absolute Error（L1损失）：

$$\mathcal{L}_{MAE} = \frac{1}{N}\sum_{i=1}^{N}\|\mathbf{y}_i - \hat{\mathbf{y}}_i\|_1 \tag{3.2}$$

对异常值鲁棒，但在零处不可微。

3.3 分类损失函数

分类任务预测离散类别标签。

3.3.1 二元交叉熵

对于二分类 ($y \in \{0, 1\}$)：

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)] \tag{3.3}$$

优势 4.

概率解释

等价于二分类的最大似然估计

即使预测自信但错误时也有强梯度

– PyTorch中的损失函数

python import torch.nn as nn
分类 criterion = nn.CrossEntropyLoss() 多分类 criterion = nn.BCEWithLogitsLoss() 二分类
回归 criterion = nn.MSELoss() MSE criterion = nn.L1Loss() MAE criterion = nn.SmoothL1Loss() Huber loss

3.4 高级损失函数

3.4.1 Focal Loss

解决密集目标检测中的类别不平衡：

$$\mathcal{L}_{\text{Focal}} = -\alpha_t(1 - \hat{y}_t)^{\gamma}\log(\hat{y}_t) \tag{3.4}$$

其中 $\gamma \geq 0$ 聚焦于难以分类的样本。

3.4.2 标签平滑

通过软化硬标签进行正则化：

$$y_{LS} = y(1 - \epsilon) + \frac{\epsilon}{K} \tag{3.5}$$

其中 $\epsilon \in [0, 1]$ 是平滑参数。

优势 5.

正则化效果

防止过度自信的预测

改善校准

3.5 选择损失函数

Table 3.1 – 损失函数选择指南

任务	推荐损失	备注
二分类	BCE	标准选择
多分类	交叉熵	可选标签平滑
回归	MSE	对异常值敏感
鲁棒回归	MAE/Huber	抗异常值
类别不平衡分类	Focal Loss	聚焦困难样本
分割	Dice + CE	处理类别不平衡

3.6 本章小结

1. **损失函数量化预测误差** 并指导网络优化。
2. **MSE** 是回归的标准但对异常值敏感。
3. **交叉熵** 是分类的标准，具有概率解释。
4. **高级损失** 如Focal Loss和Dice Loss解决特定挑战。
5. **标签平滑** 提供正则化和改善校准。

Chapter 4

神经网络架构

4.1 引言

神经网络架构定义了神经元的组织方式和连接方式，以处理信息。本章介绍三种基本神经网络架构：深度神经网络(DNN)、卷积神经网络(CNN)和循环神经网络(RNN)。

4.2 深度神经网络（DNN）

深度神经网络由多层神经元组成，信息从输入流向输出，无环路。

全连接DNN执行：

$$\mathbf{h}^{(l)} = f^{(l)}(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \tag{4.1}$$

属性：

- **通用逼近：** 可以逼近任何连续函数
- **适合：** 表格数据、基于特征的分类

4.3 卷积神经网络（CNN）

CNN通过局部连接和参数共享利用数据的空间结构。

卷积操作：

$$y[i,j] = \sum_m \sum_n w[m,n] \cdot x[i+m, j+n] \tag{4.2}$$

关键参数：

- **卷积核大小：** $K \times K$（通常 $3 \times 3, 5 \times 5$）
- **步长：** s（步长，通常1或2）
- **填充：** p（零填充控制输出大小）

输出大小：

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - K}{s} + 1 \right\rfloor \tag{4.3}$$

属性：

- **局部感受野**
- **参数共享**
- **平移等变性**
- **适合：** 图像、视频、空间数据

4.4 循环神经网络（RNN）

RNN通过维护隐藏状态处理序列数据。

对于输入序列 $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(T)}$：

$$\mathbf{h}^{(t)} = f(\mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{b}_h) \tag{4.4}$$

$$\mathbf{y}^{(t)} = g(\mathbf{W}_{hy}\mathbf{h}^{(t)} + \mathbf{b}_y) \tag{4.5}$$

属性：

- **序列处理**
- **共享参数**
- **记忆：** 隐藏状态捕获过去信息
- **适合：** 文本、时间序列、音频

4.5 长短期记忆网络（LSTM）

LSTM通过门控机制解决梯度消失问题。

LSTM门：

$$\mathbf{f}^{(t)} = \sigma(\mathbf{W}_f[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(遗忘门)} \tag{4.6}$$

$$\mathbf{i}^{(t)} = \sigma(\mathbf{W}_i[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(输入门)} \tag{4.7}$$

$$\mathbf{o}^{(t)} = \sigma(\mathbf{W}_o[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(输出门)} \tag{4.8}$$

属性：

- **选择性记忆**
- **长期依赖**
- **广泛使用**

4.6 门控循环单元（GRU）

GRU用更少的门简化了LSTM：

$$\mathbf{z}^{(t)} = \sigma(\mathbf{W}_z[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(更新门)} \tag{4.9}$$

$$\mathbf{r}^{(t)} = \sigma(\mathbf{W}_r[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(重置门)} \tag{4.10}$$

比LSTM参数更少，性能通常相当。

4.7 架构选择

4.8 本章小结

1. DNN是用于通用学习的全连接网络

Table 4.1 – 架构选择指南

架构	最佳用途	示例
DNN	表格数据	分类、回归
CNN	图像、空间数据	目标检测、分割
RNN/LSTM	序列	自然语言处理、时间序列
Transformer	长序列	翻译、生成

2. CNN通过局部连接利用空间结构
3. RNN通过循环连接处理序列数据
4. LSTM和GRU通过门控解决梯度消失问题
5. 架构选择取决于数据类型和任务

Chapter 5

神经网络核心操作

5.1 引言

本章介绍现代神经网络的基本构建块：卷积、池化和归一化操作。理解这些操作对于设计和实现有效的神经网络架构至关重要。

5.2 卷积操作

卷积是卷积神经网络（CNN）的基石。

对于输入 $\mathbf{X} \in \mathbb{R}^{H \times W}$ 和卷积核 $\mathbf{K} \in \mathbb{R}^{K \times K}$：

$$y[i,j] = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} k[m,n] \cdot x[i+m, j+n] \tag{5.1}$$

卷积参数：

- **卷积核大小**：$K \times K$（常见：$3 \times 3, 5 \times 5, 7 \times 7$）
- **步长**（s）：滑动窗口的步长
- **填充**（p）：输入周围的零填充
- **膨胀**（d）：卷积核元素之间的间距

输出大小：

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - d(K-1) - 1}{s} + 1 \right\rfloor \tag{5.2}$$

卷积优势：

- **局部感受野**
- **参数共享**
- **平移等变性**

5.3 池化操作

池化减少空间维度并提供平移不变性。

5.3.1 最大池化

$$y[i,j] = \max_{m,n \in \mathcal{N}(i,j)} x[m,n] \tag{5.3}$$

保留最强激活，提供局部平移不变性。

5.3.2 平均池化

$$y[i,j] = \frac{1}{K \cdot K} \sum_{m,n \in \mathcal{N}(i,j)} x[m,n] \tag{5.4}$$

在邻域内取平均。

5.4 归一化技术

归一化通过控制层输入分布来稳定训练。

5.4.1 批归一化（BatchNorm）

在批次维度上归一化激活：

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \tag{5.5}$$

属性：

- 减少内部协变量偏移
- 允许更高的学习率
- 提供轻微正则化

5.4.2 层归一化（LayerNorm）

在单个样本的特征维度上归一化：

$$\mu = \frac{1}{D} \sum_{i=1}^{D} x_i, \quad \sigma = \frac{1}{D} \sum_{i=1}^{D} (x_i - \mu)^2 \tag{5.6}$$

独立于批次大小，用于RNN和Transformer。

Table 5.1 – 归一化方法比较

方法	归一化维度	依赖批次	使用场景
BatchNorm	(H, W, C)	是	CNNs
LayerNorm	(H, W, N)	否	Transformers
InstanceNorm	(H, W)	否	风格迁移
GroupNorm	(H, W, G)	否	小批次CNNs

5.5 本章小结

1. 卷积通过共享权重提取局部特征
2. 池化提供空间约简和平移不变性
3. 归一化稳定训练动态

4. 现代架构将这些操作组合成标准化块
5. 操作选择取决于任务和数据特征

Chapter 6

跳跃连接与注意力机制

6.1 跳跃连接

跳跃连接（捷径）绕过层以实现深度网络中的梯度流动。

6.1.1 残差连接（ResNet）

残差学习框架：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{6.1}$$

其中 $\mathcal{F}$ 是学习的残差映射。

属性：

- **实现极深网络（100+层）的训练**
- **缓解梯度消失/爆炸问题**
- **减少退化问题**
- **易于实现，计算开销小**

如果维度匹配：$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$
如果维度不同：$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}$

6.2 注意力机制

注意力允许模型关注输入的相关部分。

6.2.1 自注意力

注意力函数：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6.2}$$

其中：

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$（查询）
- $\mathbf{K} = \mathbf{X}\mathbf{W}_K$（键）
- $\mathbf{V} = \mathbf{X}\mathbf{W}_V$（值）

缩放因子 $\sqrt{d_k}$ 防止大 d_k 时的梯度消失。
自注意力属性：

- **允许建模长期依赖**
- **计算 $O(n^2)$ 但可并行化**
- **可解释：** 注意力权重显示相关性

6.3 本章小结

1. 跳跃连接通过改善梯度流动实现深度网络
2. 残差连接是使用最广泛的跳跃连接
3. 注意力机制允许动态关注相关特征
4. 自注意力是Transformer的基础
5. 通道注意力（SE）提供高效的特征重校准

Chapter 7

优化器与学习率调度器

7.1 梯度下降优化

训练神经网络通过沿减少损失的方向更新参数来最小化损失函数。

7.1.1 随机梯度下降（SGD）

使用单个样本的梯度更新参数：

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t; \mathbf{x}^{(i_t)}, \mathbf{y}^{(i_t)}) \tag{7.1}$$

属性：

- **优点：** 快速，能逃脱局部最小值
- **缺点：** 梯度有噪声，需要较小的学习率

使用动量加速收敛：

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1-\beta)\nabla_\theta \mathcal{L}(\theta_t) \tag{7.2}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_{t+1} \tag{7.3}$$

7.2 自适应学习率方法

这些方法根据梯度历史调整每个参数的学习率。

7.2.1 Adam（自适应矩估计）

结合动量和RMSProp：

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \tag{7.4}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \tag{7.5}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t} \tag{7.6}$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \tag{7.7}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{7.8}$$

性质列表 3 (Adam属性).

- **大多数深度学习任务的默认优化器**
- **对超参数具有鲁棒性**
- **默认参数：** $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999$
- **早期时间步的偏差校正**

– PyTorch中的优化器

python import torch.optim as optim
带动量的SGD optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
Adam optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
AdamW（解耦权重衰减） optimizer = optim.AdamW(model.parameters(), lr=0.001, weight$_d ecay$ = 0.01)

7.3 AdamW（带权重衰减的Adam）

解耦的权重衰减：

$$\theta_{t+1} = \theta_t - \alpha \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon} - \alpha\lambda\theta_t \tag{7.9}$$

属性：

- **正确的L2正则化**
- **在许多任务中泛化更好**
- **Transformer的标准选择**

7.4 学习率调度

学习率调度在训练期间调整学习率。
常用调度：

- **步衰减：** $\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/T \rfloor}$
- **余弦退火：** $\alpha_t = \alpha_{min} + \frac{1}{2}(\alpha_{max} - \alpha_{min})(1 + \cos(\pi t/T))$
- **预热：** 逐渐增加学习率

性质列表 4 (预热).

- **防止开始时的大梯度**
- **对Transformer很重要**
- **常见：** 500-10000个预热步

7.5 优化器选择

Table 7.1 – 优化器选择指南

优化器	适合场景	备注
SGD + 动量	图像分类	最佳泛化
Adam	通用	默认选择
AdamW	Transformers, NLP	微调最佳
RMSProp	RNNs	循环模型良好

7.6 本章小结

1. 梯度下降是神经网络优化的基础
2. 动量通过累积速度加速收敛
3. Adam结合动量和自适应学习率
4. AdamW提供正确的权重衰减正则化
5. 学习率调度改善收敛
6. 预热对Transformer训练很重要

Chapter 8

经典网络架构

8.1 神经网络演进

深度学习的历史显示了从简单网络到越来越复杂架构的发展，每个都在前一个时代的基础上进行创新。

8.2 LeNet（1998）

Yann LeCun的LeNet是首个成功用于数字识别的CNN。

[LeNet-5架构]

1. 输入：32×32 灰度图像
2. C1：6 个 5×5 卷积核 $\rightarrow 28 \times 28 \times 6$
3. S2：2×2 平均池化 $\rightarrow 14 \times 14 \times 6$
4. C3：16 个 5×5 卷积核 $\rightarrow 10 \times 10 \times 16$
5. S4：2×2 平均池化 $\rightarrow 5 \times 5 \times 16$
6. C5：120 个 5×5 卷积核 $\rightarrow 120$
7. F6：全连接，84个神经元
8. 输出：10类（数字）

参数：约60,000

关键创新：

- 通过卷积实现权重共享
- 局部感受野
- 空间子采样（池化）

8.3 AlexNet（2012）

AlexNet以显著优势赢得ImageNet LSVRC-2012，彻底改变了计算机视觉。

参数：约6000万

关键创新：

- ReLU激活（训练更快）
- GPU训练
- 数据增强
- Dropout正则化
- 响应归一化

8.4 VGGNet（2014）

VGGNet强调深度和简洁，仅使用 3×3 卷积。
参数：约1.38亿
关键洞察：两个 3×3 卷积的参数量比一个 5×5 卷积少约28%。

8.5 GoogLeNet / Inception（2014）

引入了并行多尺度卷积的Inception模块。
关键创新：

- 多尺度特征提取
- 1×1 卷积降维
- 全局平均池化
- 辅助分类器

参数：约400万（非常高效！）

8.6 ResNet（2015）

ResNet引入了残差连接，实现了极深网络的训练。
关键创新：

- 跳跃连接实现梯度流动
- 恒等捷径
- 瓶颈设计（$1 \times 1, 3 \times 3, 1 \times 1$）
- 可实现1000+层

参数：约2500万

8.7 Transformer（2017）

Transformer用自注意力替代了循环。
关键创新：

- 自注意力替代循环
- 可并行化（不同于RNN）
- 捕获长期依赖
- 可随计算扩展

参数：65M（基础）到175B（GPT-3）及更多

Table 8.1 – 架构比较

网络	年份	参数	**Top-1准确率**
AlexNet	2012	60M	62.5%
VGG-16	2014	138M	71.5%
GoogLeNet	2014	4M	72.0%
ResNet-50	2015	25M	76.1%
ViT-B/16	2021	86M	79.9%

8.8 本章小结

1. LeNet开创了用于数字识别的CNN
2. AlexNet通过GPU训练普及了深度学习
3. VGG展示了通过 3×3 卷积实现深度
4. Inception引入了多尺度处理
5. ResNet通过跳跃连接实现了超深网络
6. Transformer通过注意力革新了NLP并影响视觉领域

Chapter 9

参数量与计算量分析

9.1 模型效率的重要性

理解模型效率对于以下方面至关重要：

- 在资源受限设备上部署
- 训练成本估算
- 模型选择和架构设计
- 基准测试和比较

关键指标：

- **参数量**：模型大小和内存占用
- **FLOPs**：计算复杂度
- **延迟**：推理时间
- **吞吐量**：每秒样本数

9.2 参数计数

参数是网络中的可学习权重和偏置。

9.2.1 全连接层

对于输入维度 d_{in} 和输出维度 d_{out}：

$$\text{权重} = d_{in} \times d_{out} \tag{9.1}$$
$$\text{偏置} = d_{out} \tag{9.2}$$
$$\text{总计} = d_{in} \times d_{out} + d_{out} \tag{9.3}$$

示例：784到256的全连接层：

$$784 \times 256 + 256 = 200,960 \text{ 个参数} \tag{9.4}$$

9.2.2 卷积层

对于输入通道 C_{in}，输出通道 C_{out}，卷积核大小 K：

$$\text{权重} = K \times K \times C_{in} \times C_{out} \tag{9.5}$$
$$\text{偏置} = C_{out} \tag{9.6}$$
$$\text{总计} = K^2 \times C_{in} \times C_{out} + C_{out} \tag{9.7}$$

9.3 FLOPs（浮点运算次数）

FLOPs衡量前向传播的计算成本。

9.3.1 卷积层FLOPs

对于输出特征图 $H_{out} \times W_{out}$：

$$\text{每个输出像素FLOPs} = K^2 \times C_{in} \times C_{out} \times 2 \tag{9.8}$$

$$\text{总FLOPs} = H_{out} \times W_{out} \times K^2 \times C_{in} \times C_{out} \times 2 \tag{9.9}$$

示例：224×224 输入，3×3 卷积，64入，128出：

$$H_{out} \times W_{out} = 224 \times 224 = 50,176 \tag{9.10}$$

$$\text{FLOPs} = 50,176 \times 3 \times 3 \times 64 \times 128 \times 2 \approx 1.17 \text{ GFLOPs} \tag{9.11}$$

9.4 实际示例

Table 9.1 – AlexNet参数统计

层	输出	参数	**FLOPs**
Conv1	$96 \times 55 \times 55$	35K	105M
Conv2	$256 \times 27 \times 27$	614K	448M
Conv3	$384 \times 13 \times 13$	885K	149M
Conv4	$384 \times 13 \times 13$	1.3M	224M
Conv5	$256 \times 13 \times 13$	884K	149M
FC6	4096	37.7M	37.7M
FC7	4096	16.8M	16.8M
FC8	1000	4.1M	4.1M
总计	-	约60M	约1.1B

9.5 迁移学习与效率

迁移学习可以显著减少所需数据和训练成本。

性质列表 5 (迁移学习优势).

- 更低的训练成本
- 可使用更小的数据集
- 收敛更快
- 通常性能更好

常见策略：

1. **冻结：** 保持预训练权重固定
2. **微调：** 以小学习率调整部分层
3. **特征提取：** 使用预训练作为特征提取器

9.6 本章小结

1. 参数量决定模型大小和内存需求
2. FLOPs衡量计算复杂度
3. MobileNet等高效架构减少两者
4. 迁移学习高效利用预训练模型
5. 理解效率指导模型选择

Chapter 10

训练过程与框架

10.1 训练循环

神经网络训练包括在数据上的迭代优化。

10.2 超参数

影响训练的关键超参数：

10.2.1 学习率

最重要的超参数。

- **过高：** 发散，损失爆炸
- **过低：** 收敛慢，陷入局部最小值
- **典型范围：** 10^{-5} 到 0.1
- **Adam默认值：** 10^{-3}
- **SGD默认值：** 10^{-2}

10.2.2 批次大小

每个训练步的样本数。

- **小批次：** 梯度更噪声，泛化更好
- **大批次：** 梯度更平滑，但需要学习率缩放
- **典型值：** 16, 32, 64, 128, 256

Table 10.1 – 超参数范围

超参数	典型范围	备注
学习率	10^{-5} 到 0.1	Adam: 10^{-3}, SGD: 10^{-2}
批次大小	16-512	取决于GPU内存
Epochs	10-1000	取决于数据集大小
权重衰减	10^{-6} 到 10^{-2}	AdamW: 10^{-2}典型
Dropout	0.1-0.5	取决于正则化需求
动量	0.9-0.999	用于SGD

10.3 正则化

防止过拟合的技术：

10.3.1 Dropout

在训练期间随机将激活置零：

$$\mathbf{y} = \mathbf{W} \cdot (\mathbf{x} \odot \mathbf{m}) \tag{10.1}$$

其中 $\mathbf{m} \sim \text{Bernoulli}(p)$ 是掩码。

属性：

- 子网络集成
- 减少共适应
- 轻微增加训练时间
- 典型dropout率：0.1-0.5

10.4 反向传播

通过计算图高效计算梯度的算法。

10.5 PyTorch框架

PyTorch是最流行的深度学习研究框架。

– PyTorch训练循环

python import torch import torch.nn as nn import torch.optim as optim from torch.utils.data import DataLoader
模型 model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.2), nn.Linear(256, 10))
损失和优化器 criterion = nn.CrossEntropyLoss() optimizer = optim.Adam(model.parameters(), lr=0.001, weight$_d ecay = 0.01)$
训练循环 device = torch.device('cuda' if torch.cuda.is$_a vailable()else'cpu')model = model.to(device)$
for epoch in range(num$_e pochs) : model.train()forbatch_i dx, (data, target)inenumerate(train_l oader) : data, target = data.to(device), target.to(device)$
optimizer.zero$_g rad()output = model(data.view(data.size(0), -1))loss = criterion(output, target)loss.backw$

10.6 CUDA与GPU训练

PyTorch使用CUDA进行GPU加速。

– PyTorch中的CUDA操作

python import torch
检查CUDA可用性 print(torch.cuda.is$_a vailable())print(torch.cuda.device_c ount())$
将张量移到GPU device = torch.device('cuda') x = torch.randn(1000, 1000).to(device) y = torch.randn(1000, 1000).to(device) z = x @ y 在GPU上进行矩阵乘法
将模型移到GPU model = MyModel().to(device)
混合精度训练 scaler = torch.cuda.amp.GradScaler() with torch.cuda.amp.autocast(): output = model(data) loss = criterion(output, target)

1. 初始化模型权重
2. 对每个epoch：
   (a) 打乱训练数据
   (b) 对每个batch：
      i. 前向传播：计算预测
      ii. 计算损失
      iii. 反向传播：计算梯度
      iv. 使用优化器更新权重
   (c) 在验证集上评估
   (d) 如果改善则保存检查点
3. 返回最佳模型

1. 前向传播：计算所有中间值
2. 计算损失 $\mathcal{L}$
3. 反向传播：
   (a) $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ 在输出处
   (b) 对每个层（逆序）：
      i. 计算 $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$
      ii. 计算 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ 用于权重更新

scaler.scale(loss).backward() scaler.step(optimizer) scaler.update()

性质列表 6 (CUDA原理).

- GPU擅长并行操作（矩阵乘法、卷积）
- CPU和GPU之间的数据传输成本高
- 批处理提高GPU利用率
- 混合精度（FP16）减少内存并增加速度

10.7 本章小结

1. 训练涉及在批次上的迭代优化
2. 超参数（学习率、批次大小）关键影响训练
3. 正则化防止过拟合
4. 反向传播高效计算梯度
5. PyTorch提供灵活的深度学习原语
6. CUDA实现训练加速

Chapter 11

数据集处理

11.1 数据收集

任何机器学习系统的基础是高质量的数据。

11.1.1 数据来源

- **公开数据集：** ImageNet、COCO、MNIST等
- **网页抓取：** 从网站自动收集数据
- **传感器：** 相机、麦克风、医疗设备
- **人工标注：** 专家标注
- **合成数据：** 从模拟生成

[数据质量] 垃圾输入，垃圾输出。高质量数据比复杂的算法更重要。

11.2 数据标注

标签使监督学习成为可能。

11.2.1 标注类型

- **分类：** 每个图像单个标签（猫 vs 狗）
- **多标签：** 每个图像多个标签
- **目标检测：** 带标签的边界框
- **语义分割：** 像素级标签
- **实例分割：** 每个对象的实例ID
- **关键点：** 感兴趣点（人脸、姿态）

11.3 数据划分

正确的数据划分对无偏评估至关重要。

11.3.1 基本划分

- **训练集：** 用于模型训练
- **验证集：** 用于超参数调优
- **测试集：** 用于最终评估（仅一次）

典型比例：

- 小数据集（¡10K）：70/10/20 或 80/10/10

- 大数据集（¿1M）：98/1/1可能足够

[测试集泄露] 永远不要在训练期间使用测试集做任何决定。这会导致对测试集的过拟合和不可靠的性能估计。

11.4 数据增强

通过转换现有数据人为增加数据集大小。

11.4.1 图像增强

- **旋转：** 随机角度旋转
- **翻转：** 水平/垂直翻转
- **裁剪：** 随机裁剪
- **颜色抖动：** 亮度、对比度、饱和度
- **缩放：** 随机缩放
- **平移：** 随机偏移
- **噪声：** 高斯噪声注入
- **Cutout：** 随机矩形遮挡
- **Mixup：** 混合两张图像和标签
- **CutMix：** 在图像之间剪切和粘贴补丁

– PyTorch中的数据增强

python import torchvision.transforms as T

train$_{t}ransform = T.Compose([T.RandomResizedCrop(224), T.RandomHorizontalFlip(p = 0.5), T.RandomRotation(15), T.ColorJitter(brightness = 0.2, contrast = 0.2, saturation = 0.2, hue = 0.1), T.ToTensor(), T.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225])])$

val$_{t}ransform = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToTensor(), T.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225])])$

11.5 数据加载

高效的数据加载对训练速度至关重要。

– PyTorch中的数据加载

python from torch.utils.data import DataLoader, Dataset

class MyDataset(Dataset): def $_{i}nit_{(self,data,labels,transform=None):self.data=dataself.labels=labelsself.transform=transform}$

def $_{l}en_{(self):returnlen(self.data)}$

def $_{g}etitem_{(self,idx):image=self.data[idx]label=self.labels[idx]}$

if self.transform: image = self.transform(image)

return image, label

创建数据加载器 train$_{l}oader = DataLoader(train_{d}ataset, batch_{s}ize = 32, shuffle = True, SGD$ftfi$num_{w}orkers = 4, vLpn^{''}pin_{m}emory = True,$ ffl$GPUdrop_{l}ast = True\Gamma$ffl$y!')$

11.6 归一化

标准化输入特征以获得更好的训练。

图像归一化：

$$x_{norm} = \frac{x - \mu}{\sigma} \tag{11.1}$$

常见ImageNet归一化：

- 均值：$[0.485, 0.456, 0.406]$
- 标准差：$[0.229, 0.224, 0.225]$

11.7 本章小结

1. 数据质量是模型性能的基础
2. 标注类型取决于任务
3. 正确的数据划分防止过拟合
4. 数据增强改善泛化
5. 高效数据加载加速训练
6. 归一化稳定训练